

并查集 + 图论—考前速查

并查集 (Union-Find / Disjoint Set)

问题： N 个元素，支持 Find(查集合) 和 Union(合集合)。
应用：等价关系、Kruskal MST、连通分量。

数据结构

```
int S[MAX]; // S[root] = -size (负数表示大小)
           // S[x] = parent (正数表示父节点索引)
```

初始化：每个元素自成一个集合

```
void init(int N) { for(int i=0;i<N;i++) S[i]=-1; }
```

Find — 路径压缩

```
int find(int x) {
    if (S[x] < 0) return x; // 是根
    return S[x] = find(S[x]); // 爹变祖宗!
}
```

效果：查找路径上所有节点直连根，后续近乎 $O(1)$ 。

四个版本对比

版本	操作	复杂度
朴素	随便挂	$\Theta(N^2)$ 最坏
US	小挂大	$O(N + M \log N)$
PC	爹变祖宗	每操作均摊接近常数
US+PC	两者都有	$O(M\alpha(M, N))$

考点

α 是反 Ackermann 函数， $\alpha \leq 4$ 在实际数据内。
Union-by-Size + 路径压缩 = 几乎常数时间！
路径压缩不能和 Union-by-Height 一起用（压缩改变高度），改用 Union-by-Rank。

Union — 按大小合并

```
void un(int a, int b) {
    a=find(a); b=find(b);
    if(a==b) return;
    if(S[a]>S[b]){int t=a;a=b;b=t;} // a更大
    S[a]+=S[b]; // 更新大小
    S[b]=a; // b挂到a
}
```

树高： $\leq \lfloor \log_2 N \rfloor + 1$ 。

$\log^* N$ (迭代对数)

$\log^* 65536 = 4, \log^* 2^{65536} = 5$ 。计算公式：反复取 $\log$ 到 ≤ 1 的次数。

Ackermann 函数

$A(0, j) = 1, A(1, 0) = 2, A(i, 0) = i + 2 (i \geq 2), A(i, j) = A(i - 1, A(i, j - 1))$ (双递归，极快增长)。 α 是其反函数——极慢增长。

图论 (Graph)

基本概念

$G = (V, E)$ 。无向图： $(v_i, v_j) = (v_j, v_i)$ 。有向图： $\langle v_i, v_j \rangle \neq \langle v_j, v_i \rangle$ 。
DAG：有向无环图。握手定理： $\sum deg(v) = 2|E|$ 。

图的表示

	邻接矩阵	邻接表
空间	$O(V ^2)$	$O(V + E)$
判相邻	$O(1)$	$O(deg)$
适合	稠密图	稀疏图

有向图补逆邻接表/多重表求入度。
加权边：矩阵存 weight，表在节点加 weight 字段。

1. 拓扑排序

目标：给 DAG 顶点线性排序，前驱在前。

```
void topsort(Graph G) {
    Queue Q; int cnt=0;
    for each V if(Indegree[V]==0) Enqueue(V,Q);
    while(!IsEmpty(Q)){
        V=Dequeue(Q); TopNum[V]++;cnt;
        for each W adj to V
            if(--Indegree[W]==0) Enqueue(W,Q);
    }
    if(cnt<NumVertex) Error("有环!");
}
```

$T = O(|V| + |E|)$ 。朴素版每次扫描找入度 $O(|V|^2)$ 。

2. AOV vs AOE

- AOV：顶点 = 活动，边 = 先后关系。
AOE：边 = 活动 (有权)，顶点 = 事件。
- $EC[j]$ = 事件 j 最早发生时间 = $\max_{i \rightarrow j}(EC[i] + C_{ij})$
 - $LC[i]$ = 事件 i 最晚发生时间 = $\min_{i \rightarrow j}(LC[j] - C_{ij})$
 - $Slack = LC - EC$, =0 在关键路径上

3. 最短路径

无权—BFS

BFS 就是无权图的最短路径。 $O(|V| + |E|)$ 。
两个版本：邻接矩阵 $O(|V|^2)$ ，邻接表 + 队列 $O(|V| + |E|)$ 。

Dijkstra

贪心。每次选 Dist 最小的未处理点，松弛所有邻接。

```
void dijkstra(int G[][MAX], int S, int N) {
    int known[MAX]={0}, dist[N];
    for(int i=0;i<N;i++) dist[i]=INF;
    dist[S]=0;
    for(;;){
        int V=-1, minD=INF;
        for(int i=0;i<N;i++)
            if(!known[i]&&dist[i]<minD)
                {minD=dist[i];V=i;}
        if(V==-1) break; known[V]=1;
        for(int W=0;W<N;W++)
            if(G[V][W] && !known[W])
                if(dist[V]+G[V][W]<dist[W])
                    dist[W]=dist[V]+G[V][W];
    }
}
```

$O(|V|^2)$ 朴素， $O(|E| \log |V|)$ 堆优化。不能有负权边！

负权边—Bellman-Ford

每轮松弛所有边，共 $|V| - 1$ 轮。 $O(|V| \cdot |E|)$ 。
第 $|V|$ 轮还能松弛 → 有负权环！

全源—Floyd-Warshall

```
for(k=0;k<N;k++)for(i=0;i<N;i++)for(j=0;j<N;j++)
    if(D[i][k]+D[k][j]<D[i][j]) D[i][j]=D[i][k]+D[k][j];
```

$O(|V|^3)$ 。适合稠密图。

无权最短路径手算

BFS 逐层扩展：初始 $dist[S]=0, known[S]=T$ 。每轮把所有刚 known 的邻接点 dist 更新为当前层 +1。

Dijkstra 手算示例

- 图有 7 个顶点 v1~v7，边权重见表。从 v1 开始。
1. v1 known: v2=2,v4=1
 2. v4 known(dist=1, 最小): v3=1+2=3,v5=1+2=3,v6=1+8=9,v7=1+10=11
 3. v2 known(dist=2): v4 已知跳过, v5=2+10=12>3 不变
 4. v3 known(dist=3): v6=3+5=8<9, 更新 v6=8
 5. v5 known(dist=3): v7=3+6=9>5, 不变
 6. v7 known(dist=5)
 7. v6 known(dist=8)
- 结果:v1(0),v2(2,v1),v3(3,v4),v4(1,v1),v5(3,v4),v6(8,v3),v7(5,v4)

4. 最小生成树 (MST)

Prim

类似 Dijkstra，Dist = 到当前树的最短距离。
朴素 $O(|V|^2)$ ，适合稠密图。

Kruskal

排序所有边 + Union-Find 判环。

$O(|E| \log |E|)$ ，适合稀疏图。

```
void kruskal(Graph G){
    T={}; sort(edges by weight);
    while(T有<|V|-1条边){
        取最小边(u,v);
        if(find(u)!=find(v)){加入T; un(u,v);}
    }
}
```

Prim vs Kruskal：稠密用 Prim($O(|V|^2)$)，稀疏用 Kruskal($O(|E| \log |E|)$)。

5. 网络流

核心技巧：残差图 + 反向边（撤销）。

- 1. 残差图中找增广路
- 2. 流量 = 路径上最小残差容量
- 3. 正向减流量，反向加（等于增加残差）
- 4. 重复直到无增广路

三种变体：BFS 找路 / 最大容量优先 / 最少边优先。

当 capmax 为小整数时， $O(|E|^2 \log |V|)$ 。

贪心不加反向边的问题：s→a→d→t 可能不是最优，需要反向边撤销。

6. DFS 应用

```
void DFS(Vertex V) {
    visited[V]=1;
    for(each W adj to V)
        if(!visited[W]) DFS(W);
}
```

$T = O(|V| + |E|)$ 。

双连通分量

Num[v] = DFS 编号，Low[v] = 通过回边能到的最小 Num。

- 树边 (Tree edge): 父 → 子
- 回边 (Back edge): 后代 → 祖先

图论复杂度速查

算法	时间复杂度	备注
拓扑排序 (队列)	$O(V + E)$	只适用于 DAG
BFS 无权最短路径	$O(V + E)$	队列实现
Dijkstra	$O(V ^2)$ 或 $O(E \log V)$	不能有负权边
Bellman-Ford	$O(V \cdot E)$	可处理负权，无负权环
Floyd-Warshall	$O(V ^3)$	全源最短路径
Prim	$O(V ^2)$ 或 $O(E \log V)$	稠密图 MST
Kruskal	$O(E \log E)$	稀疏图 MST
DFS	$O(V + E)$	递归 + 回溯
双连通分量	$O(V + E)$	DFS+Num/Low
网络流 (F-F)	$O(f \cdot E)$	f= 最大流值

- 无向图 DFS 无前向边和交叉边

关节点条件：

- v 是根且 ≥ 2 个孩子 → 关节点
- v 非根：存在孩子 w, $Low[w] \geq Num[v] \rightarrow v$ 是关节点

计算 Low：

```
Low[v] = min{ Num[v],
              Num[w]  (if (v,w) is back edge),
              Low[w]  (if (v,w) is tree edge) }
```

强连通分量 (SCC)

有向图。两次 DFS：原图 + 转置图。Tarjan 算法或 Kosaraju 算法。

欧拉回路

- 无向图欧拉回路 $\iff$ 所有顶点度数为偶数
- 无向图欧拉路径 $\iff$ 恰两个顶点度数为奇数（起点/终点）
- 有向图欧拉回路 $\iff$ 每个点 in-degree = out-degree

找回路：DFS，每条边走完删除，回溯时记录顶点。用邻接表 + 删除边。

哈密顿回路

经过每个顶点恰好一次的回路。NP 完全问题，没有多项式解。